



UNIVERSITÀ DEGLI STUDI DI FIRENZE

Scuola di Ingegneria

Corso di Laurea in Ingegneria Informatica

ChargeNet

Piattaforma per la gestione di stazioni di ricarica EV

Candidati:

Tommaso Sottili 7138541

Luca Moracci 7140812

Marco Lampronti 7147400

Corso:

Ingegneria del software

Docente:

Prof. Enrico Vicario

Anno Accademico 2025/2026

Indice

1	Introduzione	3
1.1	Descrizione	3
1.2	Stack Tecnologico	3
1.3	Semplificazioni consapevoli	4
2	Analisi dei Requisiti e Casi d'uso	5
2.1	Use Case Diagram	5
2.2	Use Case Templates	6
2.3	Mockups	10
2.4	Page Navigation Diagram	10
2.5	Package Diagram	10
2.6	Class Diagram	11
2.6.1	Domain Model	11
2.6.2	Business Layer	11
2.6.3	DAO	12
2.6.4	Presentation Layer	12
2.7	Sequence Diagram	13
2.7.1	Ciclo di vita del GridMonitor	14
2.7.2	Approvazione di una colonna	15
2.7.3	Gestione dell'evento di temperatura troppo alta (Thermal Alert)	16
2.8	Database	17
2.8.1	Schema ER	18
2.8.2	Modello Relazionale	19
3	Implementazione	21
3.1	Domain Model	21
3.1.1	Utenti	22
3.1.2	Infrastrutture	23
3.1.3	Sessioni	24
3.1.4	Finanze	25
3.1.5	Feedback	26
3.2	Business Layer	27
3.2.1	GridCluster	27
3.2.2	GridMonitor	28
3.2.3	LoadBalancerService	29
3.2.4	SessionService	30
3.2.5	StationService	31
3.2.6	Sicurezza e Validazione	31

3.2.7	Gestione Utente e Portafoglio	32
3.2.8	Gestione Feedback e Manutenzione	32
3.3	Controllers	34
3.4	ORM	35
3.5	Idiomi e Design Pattern	39
3.5.1	Observer Pattern	39
3.5.2	Strategy Pattern	39
3.5.3	Factory Method	40
3.5.4	Singleton	40
4	Testing	42
4.1	Strategia di Verifica	42
4.1.1	Tecnologie e Strumenti	42
4.2	Livello 1: Business Logic	42
4.2.1	Logica pura, senza mock — <code>ChargingStrategiesTest</code>	42
4.2.2	Singleton e reset dello stato : <code>GridClusterTest</code>	43
4.2.3	Service e gestione transazionale : <code>DriverServiceTest</code> , <code>LoadBalancerServiceTest</code>	44
4.3	Livello 2: Persistence Layer (DAO)	46
5	Demo	48

Capitolo 1

Introduzione

1.1 Descrizione

Il progetto **ChargeNet** è un sistema software pensato per la gestione di una rete di stazioni di ricarica per veicoli elettrici (EV). L'idea alla base dell'applicazione è quella di coordinare tutto ciò che ruota attorno a una sessione di ricarica: dall'aiutare l'utente a trovare una colonnina libera, fino alla gestione del pagamento, assicurandosi allo stesso tempo che l'infrastruttura elettrica non subisca danni.

Per raggiungere questo obiettivo, la piattaforma fa da punto di incontro per tre specifiche figure, fornendo a ciascuna le funzionalità necessarie:

- **Driver:** È l'utente finale. Utilizza il sistema per cercare le stazioni compatibili con la sua auto, far partire e monitorare la ricarica, e gestire i pagamenti tramite un portafoglio virtuale interno (wallet).
- **Station Operator:** È l'addetto alla manutenzione fisica. Il suo compito è controllare lo stato di salute delle colonnine: se una stazione si guasta, può metterla temporaneamente fuori servizio per evitare disagi ai Driver, e registrarla di nuovo come attiva una volta riparata.
- **Energy Manager:** È il tecnico supervisore della rete elettrica. Controlla in tempo reale il carico sui trasformatori per evitare surriscaldamenti. Se la rete è sotto stress, ha gli strumenti per abbassare la potenza erogata o limitare le ricariche (Load Balancing) in modo da proteggere l'infrastruttura.

1.2 Stack Tecnologico

- **Linguaggio:** Java, essendo un linguaggio fortemente orientato agli oggetti, ci ha permesso di modellare in modo chiaro le entità del dominio e le loro interazioni.
- **Interfaccia grafica:** JavaFX con FXML, che separa il file `.fxml` dai `Controller`.
- **Persistenza:** database relazionale PostgreSQL. In sviluppo si è usato H2 in modalità `MODE=PostgreSQL`, così da accettare la stessa sintassi SQL e rendere veloce e semplice il passaggio a PostgreSQL.
- **Accesso ai dati:** JDBC con query scritte a mano, senza ORM, per controllo esplicito della traduzione oggetti/tabelle.

- **Sicurezza:** password protette con hash BCrypt.
- **Build e dipendenze:** Maven.
- **Test:** JUnit 5 con Mockito per l'isolamento tramite mock.

1.3 Semplificazioni consapevoli

Trattandosi di un progetto universitario e viste chiare limitazioni di risorse, sono state adottate delle semplificazioni, con le quali comunque si è cercato di replicare un sistema effettivamente pronto all'utilizzo nel mondo reale. Seguono alcune delle scelte più significative, che hanno permesso di ridurre la complessità senza compromettere la logica del sistema:

- La geolocalizzazione non deriva da un indirizzo reale ma dal quartiere scelto (per le colonnine) o da una lettura GPS simulata (per il Driver); in produzione si userebbe un servizio di geocoding.
- La distanza tra due punti è calcolata con formula euclidea, più che sufficiente all'unico scopo di ordinare le colonnine per vicinanza.
- Non esiste un sistema di pagamento verso l'esterno: ricariche del portafoglio e guadagni sono movimenti interni.
- I coefficienti di calore dei trasformatori sono tarati per rendere il surriscaldamento osservabile in un tempo breve. Nella realtà un trasformatore si surriscalda per l'effetto cumulativo di molte ricariche contemporanee sullo stesso ramo; nella nostra applicazione, pensata per un solo utente alla volta, non è possibile simulare più sessioni in parallelo per raggiungere lo stesso effetto. Per questo motivo il calore generato da una singola sessione è stato aumentato, così da poter osservare l'intero meccanismo (accumulo, soglia, notifica, reazione) anche con una sola ricarica attiva, senza alterarne la logica.

Capitolo 2

Analisi dei Requisiti e Casi d'uso

Questo capitolo descrive il processo di progettazione del sistema, seguendo un approccio che va dall'esterno verso l'interno. Si parte dall'analisi di come gli utenti interagiscono con l'applicazione, definendo i requisiti e l'interfaccia grafica, per poi scendere progressivamente nei dettagli tecnici dell'architettura software e della struttura del database.

2.1 Use Case Diagram

Il diagramma dei casi d'uso offre una visione d'insieme delle funzionalità del sistema. Serve a mappare visivamente quali azioni possono compiere i tre attori principali (Driver, Station Operator ed Energy Manager) all'interno della piattaforma.

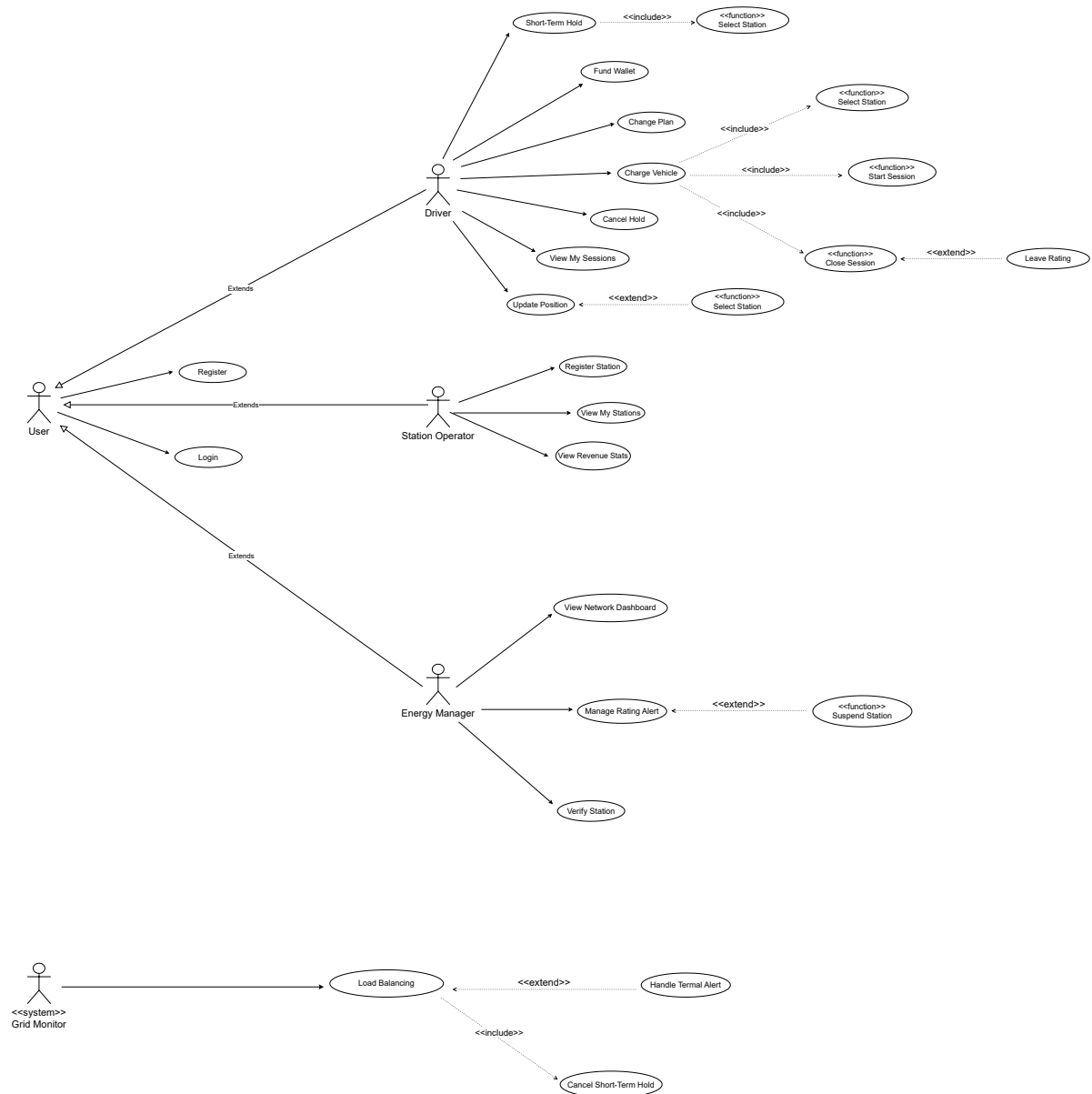


Figura 2.1: Diagramma dei casi d'uso

2.2 Use Case Templates

Per evitare ambiguità, i casi d'uso più importanti sono stati documentati nel dettaglio attraverso dei template strutturati. Questi schemi descrivono passo dopo passo il flusso degli eventi, le precondizioni necessarie e i possibili scenari di errore.

Tabella 2.1: Caso d'Uso: Ricarica Veicolo

Campo	Contenuto
Nome	Ricarica Veicolo (Charge Vehicle)
Livello	Obiettivo Utente (User Goal)
Descrizione	Il Conducente avvia una sessione di ricarica su una stazione selezionata, scegliendo una specifica strategia di ricarica (Fast o Eco). La sessione viene eseguita in tempo reale: il Grid Monitor la fa avanzare e l'interfaccia utente (Live Session UI) ne osserva lo stato tramite polling.
Pre-condizioni	Il Conducente ha effettuato l'accesso. Il saldo del Portafoglio (Wallet) del Conducente copre almeno il costo di 5 kWh presso la stazione selezionata, calcolato per il piano del Conducente (<code>station.priceFor(5.0, driver)</code>) e la strategia selezionata. La stazione selezionata è ACTIVE (oppure RESERVED dal Conducente) ed è compatibile con il connettore del veicolo.
Post-condizioni	La sessione è COMPLETED , il costo (detratto in modo incrementale per ogni tick durante la ricarica) è interamente addebitato sul Portafoglio, viene registrata una Transazione di tipo CHARGE e la stazione torna allo stato ACTIVE .
Attori	Conducente, Grid Monitor (Sistema)
Unit Test	<code>SessionServiceTest</code>

Continua nella pagina successiva...

Tabella 2.1 – Continua dalla pagina precedente

Campo	Contenuto
Flusso Principale	1. Il Conducente seleziona una stazione dalla lista delle stazioni disponibili e compatibili più vicine (ACTIVE, o RESERVED da lui stesso). 2. Il Conducente apre la Live Session per quella stazione. 3. Il Conducente sceglie una strategia di ricarica: Fast Charge o Eco Charge. 4. Il Conducente inserisce la percentuale di batteria attuale del veicolo. 5. Il Conducente preme Start; il sistema valida il saldo del Portafoglio e la disponibilità della stazione, imposta la stazione su BUSY e apre la sessione. 6. Il Grid Monitor, ogni 5s, aggiorna batteria e kWh erogati, accumula il calore sul trasformatore di alimentazione e deduce i fondi in base alla strategia (lo sconto del piano si applica solo alla quota tariffaria della piattaforma). 7. La Live Session UI interroga (poll) lo stato della sessione e riflette in tempo reale batteria, kWh, costo attuale e Portafoglio. 8. Il Conducente preme Stop (oppure la sessione si auto-completa quando la batteria raggiunge il 100%). 9. Il sistema calcola i kWh totali erogati, il costo finale e registra la Transazione. 10. Il sistema reimposta lo stato della stazione su ACTIVE e mostra il popup di valutazione (rating).
Flussi Alternativi	1. Stazioni non disponibili nascoste: 1.1. Le stazioni BUSY, OVERLOADED, incompatibili o RESERVED da altri non sono mostrate in lista, impedendo al Conducente di selezionare una stazione non disponibile. 5. Saldo insufficiente: 5.1. Se il saldo è inferiore al costo di 5 kWh, il sistema nega l'avvio e mostra: "Credito insufficiente per avviare la sessione." (<code>InsufficientBalanceException</code>). 6-7. Allarme Termico (Observer attivato): 6.1. Se la temperatura del trasformatore supera i 90°C, il <code>LoadBalancerService</code> (Observer) imposta tutte le stazioni su quel trasformatore su OVERLOADED e chiude forzatamente le relative sessioni attive. 6.2. La sessione diventa INTERRUPTED_THERMAL e la UI mostra: "Arresto di emergenza: surriscaldamento del sistema". 6.3. Il sistema elabora un rimborso automatico per i kWh pagati ma non erogati.

Campo	Contenuto
Nome	Prenotazione a Breve Termine (Short-Term Hold)
Livello	Obiettivo Utente (User Goal)
Descrizione	Il Conducente prenota temporaneamente una stazione di ricarica ACTIVE per 15 minuti per assicurarne la disponibilità al suo arrivo fisico.
Pre-condizioni	• Il Conducente ha effettuato l'accesso. • Il Conducente non ha già un'altra prenotazione attiva. • La stazione di destinazione è ACTIVE ed è compatibile con il connettore del Conducente.
Post-condizioni	Lo stato della stazione viene modificato in RESERVED , reserved_by_id viene impostato sul Conducente e expiration_timestamp viene impostato all'orario attuale più 15 minuti. La stazione prenotata rimane visibile al Conducente che ha effettuato la prenotazione (contrassegnata come RESERVED) e viene nascosta a tutti gli altri Conducenti.
Attori	Conducente
Unit Test	StationServiceTest
Flusso Principale	1. Il Conducente seleziona una stazione ACTIVE dalla lista delle stazioni disponibili. 2. Il Conducente preme Smart Book per confermare la prenotazione. 3. Il sistema verifica che la stazione sia ancora ACTIVE (controllo di concorrenza). 4. Il sistema imposta lo stato della stazione su RESERVED e reserved_by_id sul Conducente. 5. Il sistema imposta expiration_timestamp all'orario attuale più 15 minuti. 6. Il sistema mostra un messaggio di successo; la stazione appare ora come RESERVED nella lista del Conducente, offrendo solo l'opzione Charge Now.
Flussi Alternativi	3. Stazione non disponibile (Conflitto di concorrenza): 3.1. Se la stazione è stata prenotata o occupata da un altro utente nel frattempo, il sistema nega la prenotazione. 3.2. Il sistema mostra il messaggio: "Stazione non più disponibile". 3.3. Ritorno al passo 1. Nota sulla Scadenza: • Se trascorrono i 15 minuti senza che il Conducente avvii una sessione, il Grid Monitor ripristina automaticamente la stazione su ACTIVE (gestito dal caso d'uso Expire Short-Term Holds).

Tabella 2.2: Caso d'Uso: Prenotazione a Breve Termine

2.3 Mockups

In questa fase sono stati realizzati i mockup dell'interfaccia grafica per verificare la User Experience e garantire che tutte le funzionalità descritte nei casi d'uso siano accessibili in modo immediato ed intuitivo.

2.4 Page Navigation Diagram

Il diagramma di navigazione illustra in che modo le diverse schermate dell'applicazione sono collegate tra loro. Mostra il percorso che l'utente compie muovendosi all'interno del sistema in base alle scelte effettuate o ai pulsanti premuti.

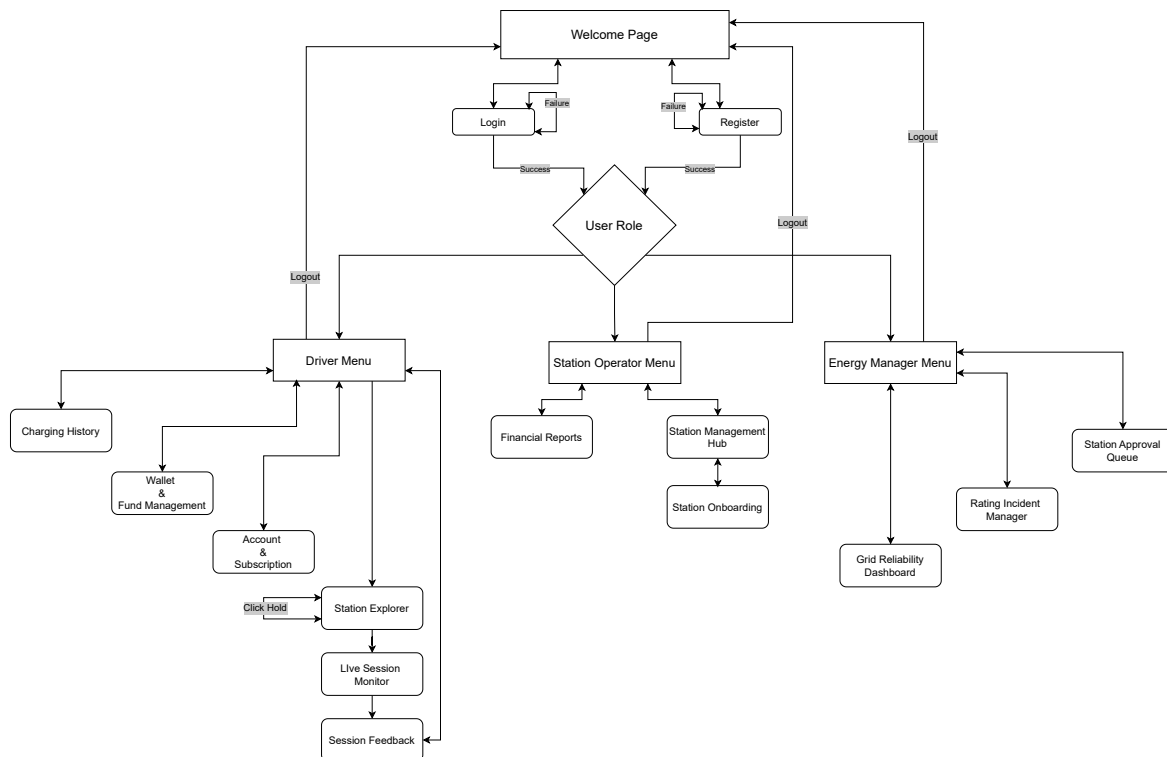


Figura 2.2: Diagramma di navigazione tra le pagine dell'applicazione.

2.5 Package Diagram

Passando all'architettura interna del software, il diagramma dei pacchetti mostra come il codice è stato diviso in moduli logici (come ad esempio la logica di business, l'interfaccia e l'accesso ai dati). L'obiettivo è mantenere il progetto ordinato e limitare le dipendenze tra i vari componenti.

2.6 Class Diagram

Il sistema è stato progettato con un'architettura a strati per mantenere alta coesione e basso accoppiamento (*high coesion and low coupling*). Nel seguente paragrafo sono approfondite le classi principali:

2.6.1 Domain Model

Questo diagramma rappresenta il Domain Model, cioè le entità principali su cui si basa il sistema: utenti, colonnine, trasformatori, sessioni di ricarica e gli oggetti ad esse collegati. Una descrizione più dettagliata sul Domain Model è presente nel paragrafo 3.1

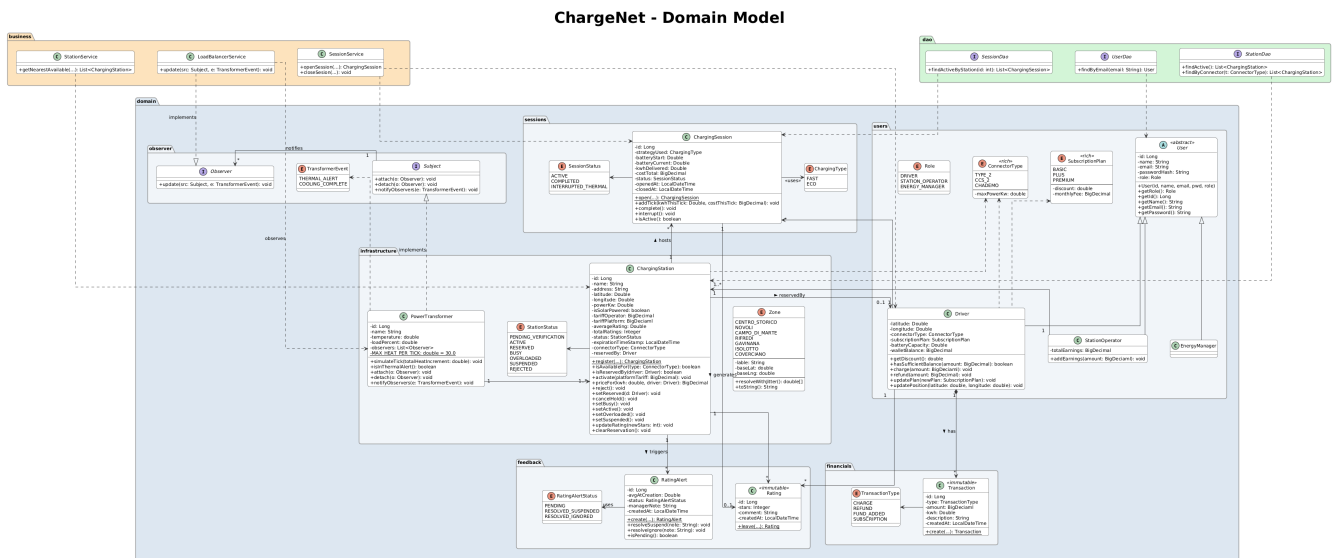


Figura 2.3: Domain Model, diagramma di classe

2.6.2 Business Layer

Il seguente diagramma mostra il Business Layer, il livello che contiene la logica applicativa del sistema. Qui si trovano i servizi che coordinano le entità del Domain Model, le strategie di ricarica e i componenti che gestiscono la simulazione della rete. Il livello è approfondito nel paragrafo 3.2.

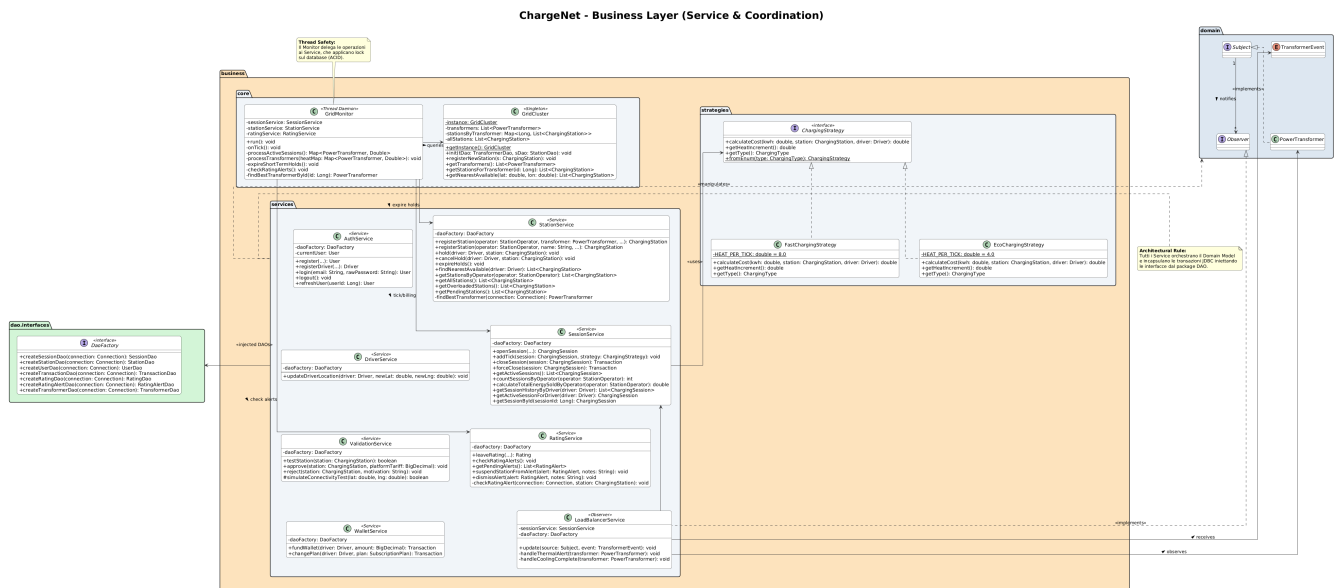


Figura 2.4: Business Layer, diagramma di classe

2.6.3 DAO

Di seguito è mostrato il diagramma del livello DAO, responsabile della comunicazione con il database. Il suo compito è tradurre le entità del Domain Model in dati salvabili e recuperabili, garantendone quindi la persistenza

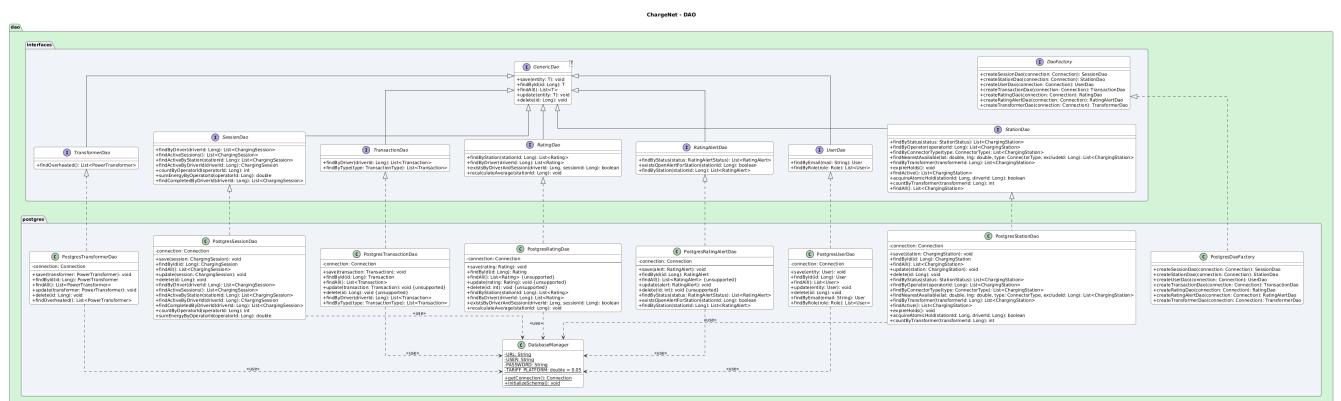


Figura 2.5: DAO, diagramma di classe

2.6.4 Presentation Layer

Il seguente diagramma rappresenta il presentation layer, cioè l'interfaccia grafica dell'applicazione realizzata in JavaFX. Comprende i controller delle varie schermate, che si occupano di mostrare i dati e raccogliere le azioni dell'utente, appoggiandosi ai servizi del livello di business.

2.7.1 Ciclo di vita del GridMonitor

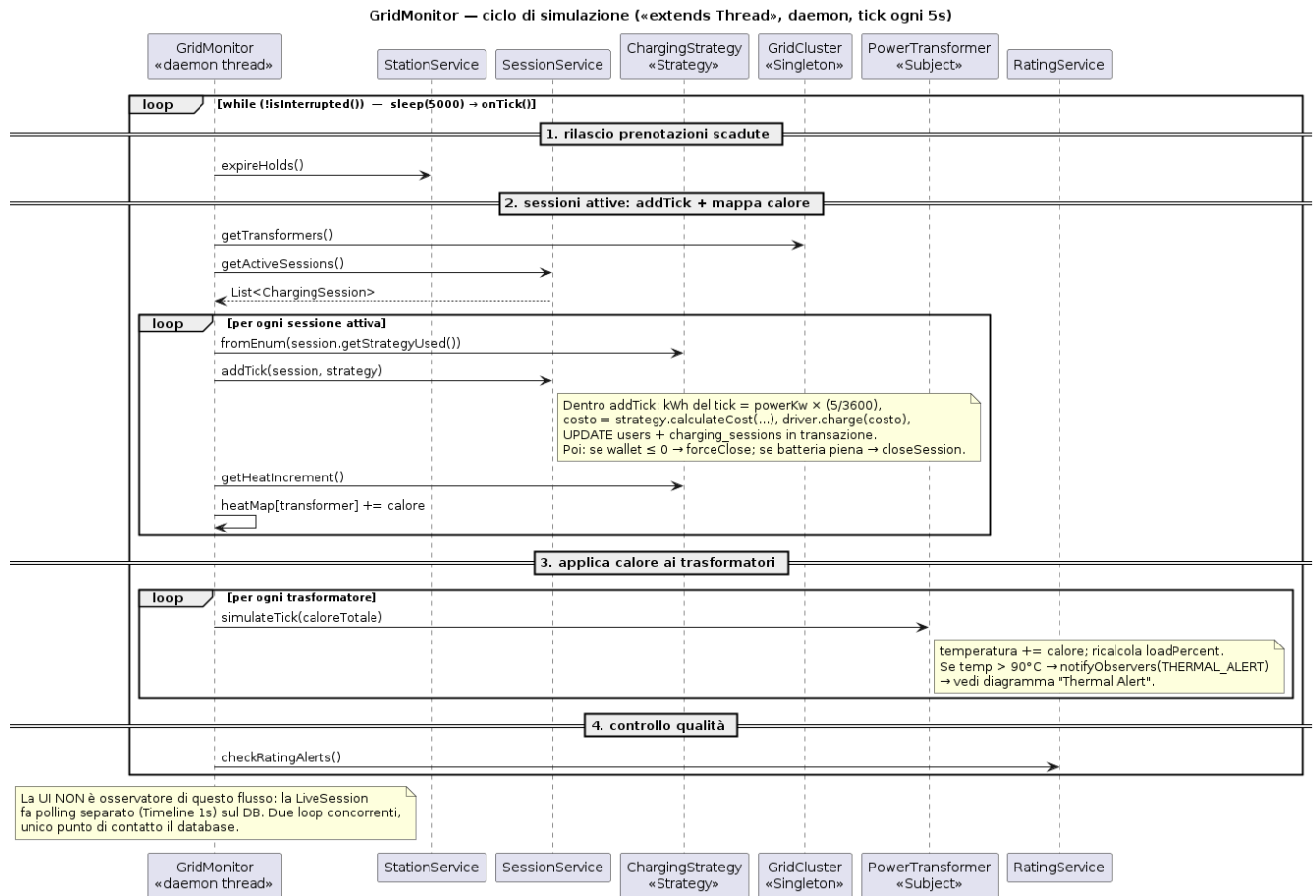


Figura 2.7: Diagramma di sequenza del ciclo di vita del GridMonitor.

Il **GridMonitor** (Fig. 2.7) è il thread daemon che scandisce l'intera simulazione della rete elettrica con un tick fisso di 5 secondi, ed è di fatto l'unico componente attivo di propria iniziativa nel sistema: tutti gli altri servizi vengono invocati su richiesta. Ad ogni ciclo il monitor libera prima le prenotazioni scadute tramite **StationService.expireHolds()**, poi recupera da **SessionService** l'elenco delle sessioni attive e, per ciascuna, seleziona la **ChargingStrategy** corrispondente per calcolare kWh erogati e costo del tick, aggiornando in un'unica transazione sia il wallet del Driver sia lo stato della sessione. Lo stesso tick genera un incremento di calore che viene accumulato per trasformatore e applicato a fine ciclo a ciascun **PowerTransformer**: se la temperatura supera i 90°C viene notificato l'evento **THERMAL_ALERT** agli observer registrati, dando origine al flusso descritto in Fig. 2.9. Infine il monitor invoca il controllo di qualità del **RatingService**. È importante osservare che la UI non partecipa a questo ciclo come observer: lo stato delle sessioni viene mostrato tramite un polling separato sul database, in modo tale che i due flussi restino disaccoppiati e l'unico punto di sincronizzazione sia la persistenza.

2.7.2 Approvazione di una colonnina

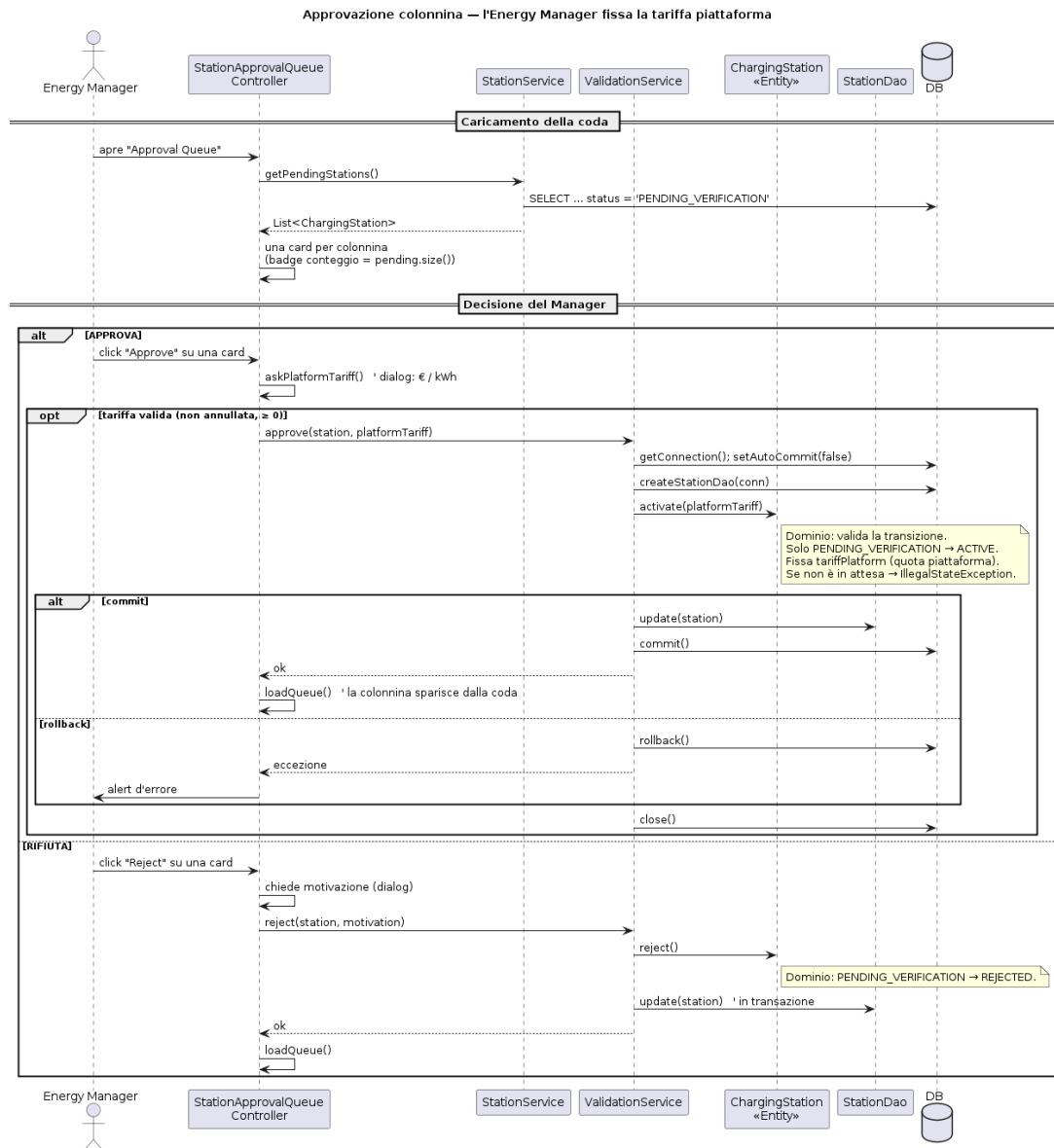


Figura 2.8: Diagramma di sequenza dell'approvazione di una colonnina da parte dell'Energy Manager.

Il flusso di approvazione (Fig. 2.8) regola l'ingresso in esercizio di una nuova colonnina proposta da uno Station Operator. L'Energy Manager apre la coda delle richieste in stato `PENDING_VERIFICATION`, popolata da `StationService.getPendingStations()`, e per ciascuna voce può approvare o rifiutare. In caso di approvazione il controller richiede la tariffa di piattaforma da applicare in €/kWh e delega la transizione alla `ValidationService`, che apre esplicitamente una connessione con `setAutoCommit(false)` prima di invocare `activate()` sul dominio: è la `ChargingStation` stessa a validare che la transizione sia legittima solo a partire da `PENDING_VERIFICATION`, sollevando un'eccezione altrimenti, mentre il servizio si occupa unicamente di orchestrare la persistenza e il commit o il rollback della transazione in caso di errore. La stessa separazione si ritrova nel percorso di rifiuto, dove il dominio applica la transizione a `REJECTED` dopo l'acquisizione

della motivazione da parte del Manager. In entrambi i casi la responsabilità di stabilire se un cambio di stato è ammesso resta interamente nel Domain Model, mentre il Business Layer si limita a decidere quando invocarla e a garantirne l'atomicità della persistenza.

2.7.3 Gestione dell'evento di temperatura troppo alta (Thermal Alert)

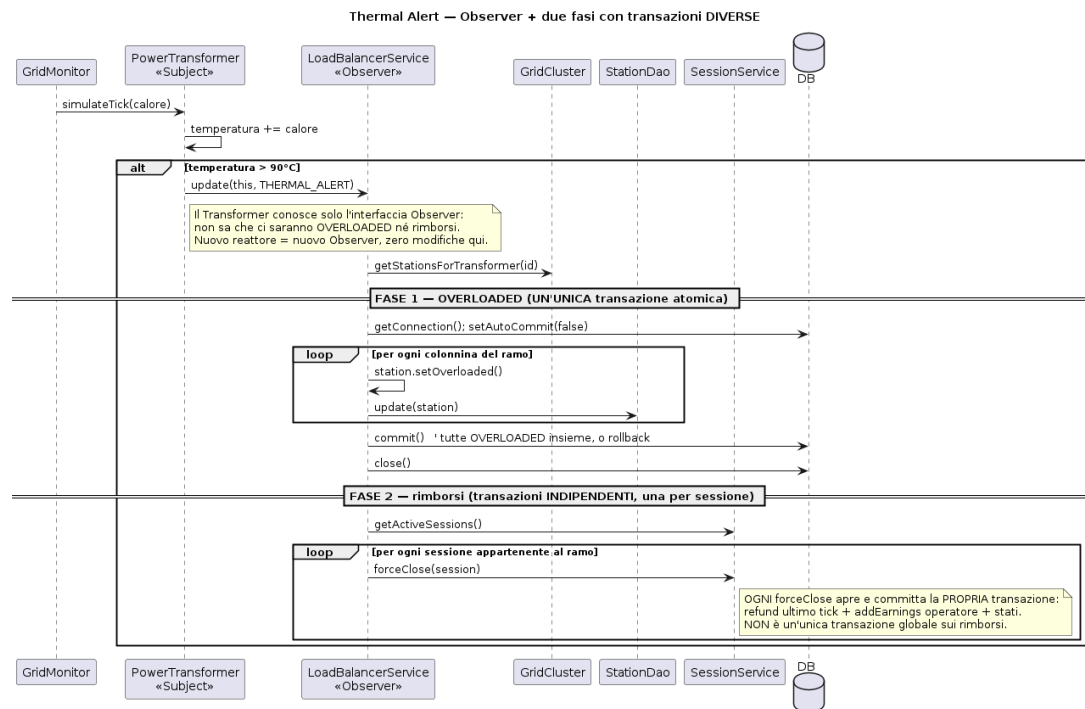


Figura 2.9: Diagramma di sequenza della gestione di una temperatura troppo alta (Thermal Alert).

Il diagramma di Thermal Alert (Fig. 2.9) mostra cosa succede quando un trasformatore si surriscalda. Durante il tick del `GridMonitor`, il `PowerTransformer` riceve il nuovo carico di calore e aggiorna la propria temperatura interna. Se questa supera i 90°C, il trasformatore avvisa il `LoadBalancerService`, che è l'unico componente incaricato di reagire all'emergenza: il trasformatore stesso non sa cosa succederà dopo, si limita a segnalare il problema.

A quel punto il `LoadBalancerService` interviene in due passaggi separati. Nel primo, blocca subito tutte le colonnine collegate a quel trasformatore mettendole in stato `OVERLOADED`, per evitare che qualcuno continui a ricaricare su un ramo elettrico in sovraccarico: questo blocco viene fatto tutto insieme, colonnina per colonnina, come un'unica operazione che riesce o fallisce nel suo complesso. Nel secondo passaggio, il sistema chiude una per una tutte le sessioni di ricarica che erano attive su quelle colonnine, rimborsando ogni Driver coinvolto tramite `SessionService.forceClose()`. Qui però ogni chiusura viene trattata singolarmente: se il rimborso di una sessione dovesse fallire, le altre sessioni verrebbero comunque chiuse e rimborsate correttamente, e soprattutto le colonnine resterebbero comunque bloccate in sicurezza, indipendentemente da come va il rimborso.

2.8 Database

La gestione dei dati di ChargeNet è affidata a un database relazionale PostgreSQL. Inizialmente, per velocizzare il lavoro, si è scelto di lavorare inizialmente con H2. Questo ha permesso di scrivere query e testare la parte applicativa fin da subito, per poi occuparsi successivamente dell'installazione e configurazione del database reale, passaggio che è stato comunque rapido visto che non è stato necessario modificare la sintassi SQL (H2 è stato usato in modalità `MODE=PostgreSQL`)

2.8.1 Schema ER

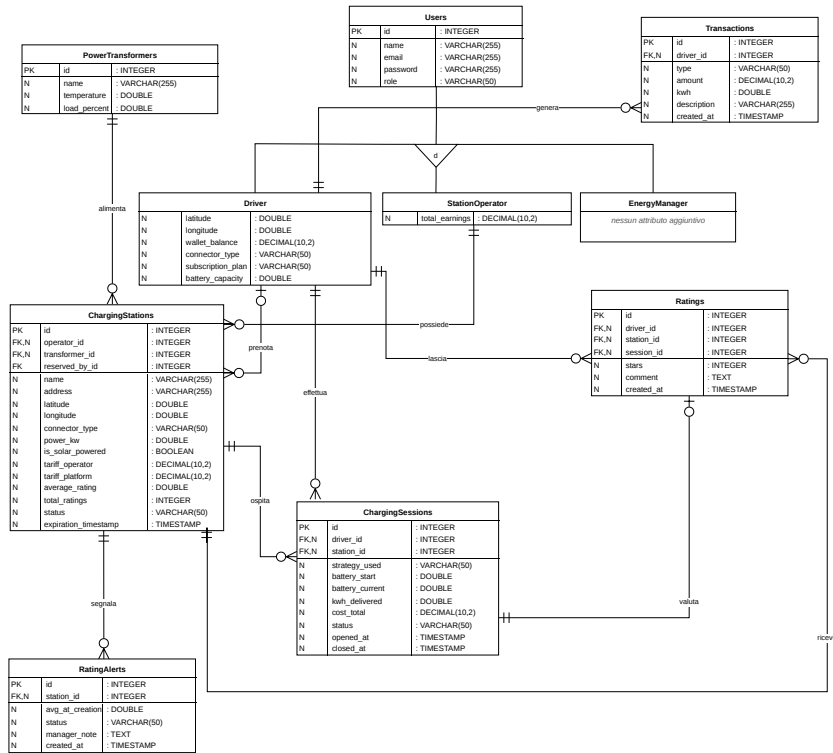


Figura 2.10: Schema Entità-Relazione del database ChargeNet.

Lo schema (Fig. 2.10) riflette la stessa gerarchia già introdotta nel Domain Model: **Users** è un'entità generica che rappresenta ciò che ogni utente ha in comune (identità, credenziali, ruolo), che viene specializzata in tre sottotipi, **Driver**, **StationOperator** ed **EnergyManager**, ciascuno con i propri attributi specifici. Un utente appartiene esattamente a uno dei tre sottotipi, mai a più di uno contemporaneamente, infatti la colonna **role** può avere un solo valore.

Le relazioni principali partono dallo specifico sottotipo e non genericamente da **Users**:

- Un **PowerTransformer** alimenta più **ChargingStation**.
- Uno **StationOperator** possiede più **ChargingStation**.
- Un **Driver** può prenotare temporaneamente una colonnina, avviare più **ChargingSession**, generare più **Transaction** e lasciare più **Rating**.
- Una **ChargingStation** ospita più **ChargingSession** e può ricevere più **Rating** o generare più **RatingAlert**.
- Ogni sessione può generare al più una **Rating**.
- **EnergyManager** non compare in nessuna relazione dello schema: il suo ruolo (approvare colonnine, gestire gli alert di qualità) non richiede di possedere o generare entità proprie, ma di agire su entità già esistenti.

2.8.2 Modello Relazionale

La traduzione delle entità semplici segue le regole standard: ogni entità diventa una tabella, ogni relazione (1,n) diventa una chiave esterna sul lato "molti". La generalizzazione invece è stata tradotta nel seguente modo: un'unica tabella con tutte le colonne di tutti i sottotipi, dove le colonne che non riguardano una riga restano NULL, e una colonna discriminatore indica il sottotipo reale.

```
Users(id, name, email, password, role, latitude, longitude, wallet_balance,  
      connector_type, subscription_plan, battery_capacity, total_earnings)
```

```
PowerTransformers(id, name, temperature, load_percent)
```

```
ChargingStations(id, operator_id, transformer_id, name, address,  
                 latitude, longitude, connector_type, power_kw, is_solar_powered,  
                 tariff_operator, tariff_platform, average_rating, total_ratings,  
                 status, reserved_by_id*, expiration_timestamp)
```

```
ChargingSessions(id, driver_id, station_id, strategy_used,  
                 battery_start, battery_current, kwh_delivered, cost_total,  
                 status,  
                 opened_at, closed_at*)
```

```
Transactions(id, driver_id, type, amount, kwh, description, created_at)
```

```
Ratings(id, driver_id, station_id, session_id, stars,  
        comment, created_at)    UNIQUE(driver_id, session_id)
```

```
RatingAlerts(id, station_id, avg_at_creation, status, manager_note*,  
             created_at)
```

dove:

`operator_id, reserved_by_id` si riferisce a `Users(id)`
`transformer_id` si riferisce a `PowerTransformers(id)`
`driver_id` si riferisce a `Users(id)`
`station_id` si riferisce a `ChargingStations(id)`
`session_id` si riferisce a `ChargingSessions(id)`

Legenda della notazione

attributo — Chiave Primaria (PK) della tabella.

attributo — Chiave Esterna (FK) verso un'altra tabella.

* — Attributo opzionale (NULL ammesso); tutti gli altri attributi sono obbligatori (NOT NULL).

UNIQUE(...) — Vincolo di unicità su una combinazione di più colonne.

La colonna `role` in `Users` dice se la riga rappresenta un `Driver`, uno `StationOperator` o un `EnergyManager`. Colonne come `wallet_balance` (`Driver`) o `total_earnings` (`StationOperator`) esistono per tutte le righe della tabella, ma restano `NULL` per i sottotipi a cui non appartengono. Per esempio ovviamente non ha senso che `EnergyManager` abbia un valore diverso da `null` nell'attributo `subscription_plan`. È lo stesso meccanismo già visto nel paragrafo ORM riguardo il metodo `extractUserFromResultSet()`, che legge la colonna `role` e ricostruisce l'oggetto Java del sottotipo corretto.

Per quanto riguarda invece le due FK verso `Users` in `ChargingStations` (`operator_id` e `reserved_by_id`) modellano due relazioni indipendenti — chi possiede la colonnina e chi la sta eventualmente prenotando, ed è proprio per questo che solo la seconda può essere nulla.

Il vincolo `UNIQUE (driver_id, session_id)` sulla tabella `Ratings` fa sì che anche a livello di database valga la regola che un `Driver` può lasciare una sola valutazione per ciascuna sessione.

Infine, diversi attributi numerici sono protetti da vincoli `CHECK` direttamente nello schema (es. `stars` tra 1 e 5, `wallet_balance` sempre ≥ 0): questi vincoli sono una seconda difesa rispetto alla validazione già presente nel Domain Model, indipendente da eventuali bug nel codice Java.

E' presente un limite, conseguenza della generalizzazione utilizzata: la foreign key `operator_id` garantisce solo che l'id referenziato esista in `Users`, non che quella riga appartenga effettivamente a uno `StationOperator`, lo stesso vale per `driver_id` rispetto ai `Driver`. Questo vincolo di coerenza tra ruolo e utilizzo non è esprimibile a livello di schema relazionale standard, tuttavia è comunque garantito nel codice Java. Infatti il metodo `StationService.registerStation()` accetta come parametro un `StationOperator`, non un generico `User`: un `Driver` non potrebbe essere passato a quel metodo nemmeno volendo, perché il codice non compilerebbe. La sicurezza che manca allo schema relazionale è quindi comunque garantita ma non appunto da quest'ultimo.

Capitolo 3

Implementazione

Questo capitolo descrive le scelte implementative e l'architettura interna di ChargeNet. Per mantenere la trattazione chiara, non verrà mostrato l'intero codice sorgente, ma ci si concentrerà sull'organizzazione dei componenti principali e sui frammenti di codice che illustrano le logiche più complesse e i pattern utilizzati.

Architettura del Sistema: I livelli principali sono tre: Domain, Business e Controller. L'interazione con il database PostgreSQL è gestita tramite il pattern Data Access Object (DAO), che nasconde la complessità delle query SQL al resto dell'applicazione.

3.1 Domain Model

Per avere maggiore ordine abbiamo suddiviso il Domain Model in cinque pacchetti principali, ognuno con una responsabilità specifica, raggruppando il codice in base alla finalità e al contesto di utilizzo. Questa struttura modulare facilita la manutenzione e l'estensione del sistema, permettendo di isolare le modifiche a specifici ambiti senza impattare l'intero progetto, oltre che a rendere più semplice la comprensione del codice per il lettore.

Di seguito sono elencati i cinque pacchetti principali, con una breve descrizione delle loro responsabilità, che verranno approfondite nel proseguo del paragrafo:

- **Utenti:** Definisce gli attori del sistema. Ci sono tre profili principali (*Driver*, *StationOperator* ed *EnergyManager*), che estendono tutti una classe madre comune per semplificare le operazioni di login e registrazione.
- **Infrastrutture:** Rappresenta le parti fisiche della rete. Contiene le colonnine (*ChargingStation*) e i trasformatori (*PowerTransformer*), e serve a gestire parametri come la potenza massima erogabile o lo stato di occupazione.
- **Sessioni:** È il collegamento pratico tra l'utente e l'infrastruttura. La classe centrale è *ChargingSession*, che si occupa di far partire la ricarica e di calcolare quanti kWh vengono consumati col passare del tempo.
- **Finanze:** Si occupa dei pagamenti e del saldo degli utenti. Gestisce le ricevute (*Transaction*) create dopo una ricarica, un rimborso o il pagamento di un abbonamento, facendo in modo che lo storico contabile non possa essere modificato per sbaglio.

- **Feedback:** Gestisce le recensioni. Raccoglie i voti (*Rating*) lasciati dai guidatori e fa scattare in automatico un avviso di manutenzione (*RatingAlert*) se la media di una stazione si abbassa troppo.

Nei prossimi paragrafi vedremo più nel dettaglio alcune delle soluzioni implementative che abbiamo adottato in questi cinque moduli.

3.1.1 Utenti

Questo modulo definisce le identità degli attori tramite un'architettura basata sull'ereditarietà. Dalla classe astratta **User** derivano i tre profili specifici (**Driver**, **StationOperator** ed **EnergyManager**), ognuno con i propri attributi.

Per rispettare il principio dell'incapsulamento, la logica di business è integrata direttamente nelle entità. Un esempio pratico è la gestione economica del **Driver**: quando c'è da scalare del credito, è l'oggetto stesso a fare i controlli sul proprio saldo.

Listing 3.1: Controllo del saldo nel metodo charge()

```
public void charge(BigDecimal amount) {
    if (!hasSufficientBalance(amount)) {
        throw new InsufficientBalanceException("Saldo insufficiente");
    }
    this.walletBalance = this.walletBalance.subtract(amount);
}
```

Questo approccio torna utile anche per operazioni più delicate. Il cambio di piano, ad esempio, non addebita il canone intero del nuovo piano, ma solo la differenza rispetto a quello attuale, e blocca i downgrade riusando comunque charge() per il controllo del saldo:

Listing 3.2: Aggiornamento del piano tariffario

```
public BigDecimal updatePlan(SubscriptionPlan newPlan) {
    BigDecimal currentFee = (this.subscriptionPlan != null)
        ? this.subscriptionPlan.getMonthlyFee() : BigDecimal.ZERO;
    BigDecimal delta = newPlan.getMonthlyFee().subtract(currentFee)
        .setScale(2, RoundingMode.HALF_UP);

    if (delta.compareTo(BigDecimal.ZERO) < 0) {
        throw new IllegalArgumentException(
            "Non e possibile passare a un piano inferiore durante il periodo attivo.");
    }
    if (delta.compareTo(BigDecimal.ZERO) > 0) {
        charge(delta);
    }
    this.subscriptionPlan = newPlan;
    return delta;
}
```

Per il collegamento con il database, invece di passare per i costruttori standard (che farebbero scattare validazioni inutili in fase di caricamento), abbiamo dotato ogni entità di un metodo `reconstitute()`, pensato esclusivamente per ripristinare in memoria i dati letti da PostgreSQL.

3.1.2 Infrastrutture

L'obiettivo di questo pacchetto è tradurre in codice i limiti fisici del mondo reale. Per la classe `ChargingStation` era fondamentale impedire la creazione di oggetti con parametri incoerenti o pericolosi.

Abbiamo quindi nascosto i costruttori, obbligando il resto del sistema a passare per il Factory Method `register()`. In questa fase la stazione esegue quattro controlli in sequenza: coordinate geografiche valide, potenza compresa tra 7 e 350 kW, potenza compatibile col limite fisico del connettore, e tariffa dell'operatore positiva. Basta che uno fallisca per bloccare subito la creazione.

Listing 3.3: Factory Method per la registrazione della stazione

```
public static ChargingStation register(StationOperator operator,
    PowerTransformer transformer,
                                   String name, String address
                                   , Double lat, Double lng
                                   ,
                                   ConnectorType connectorType
                                   , Double powerKw,
                                   Boolean isSolar, BigDecimal
                                   tariff) {
    if (lat < -90.0 || lat > 90.0) {
        throw new InvalidStationParametersException("Latitudine non
            valida");
    }
    if (lng < -180.0 || lng > 180.0) {
        throw new InvalidStationParametersException("Longitudine
            non valida");
    }
    if (powerKw < 7.0 || powerKw > 350.0) {
        throw new InvalidStationParametersException("Potenza non
            valida. Deve essere compresa tra 7 kW e 350 kW.");
    }
    if (powerKw > connectorType.getMaxPowerKw()) {
        throw new InvalidStationParametersException("Errore di
            sicurezza: La potenza richiesta (" + powerKw +
            " kW) supera il limite fisico supportato dal
            connettore " + connectorType.name() +
            " (" + connectorType.getMaxPowerKw() + " kW).");
    }
    if (tariff.compareTo(BigDecimal.ZERO) <= 0) {
        throw new InvalidStationParametersException("La tariffa
            dell'operatore deve essere strettamente positiva.");
    }
    return new ChargingStation(operator, transformer, name, address
        , lat, lng, connectorType, powerKw, isSolar, tariff);
}
```

L'altro parte fondamentale del pacchetto è il **PowerTransformer**, che fa da base per il sistema di Load Balancing. Piuttosto che avere un servizio esterno che controlla continuamente i trasformatori, abbiamo reso il trasformatore intelligente applicando il pattern Observer. A ogni ciclo di simulazione, ricalcola la sua temperatura interna; se supera la soglia critica dei 90 gradi, genera autonomamente un **THERMAL_ALERT** per avvisare la rete, senza bisogno di sapere chi o come gestirà effettivamente l'emergenza(vedasi paragrafo 3.5.1).

3.1.3 Sessioni

Il pacchetto racchiude la logica legata all'erogazione dell'energia. La classe centrale, **ChargingSession**, non si limita a registrare passivamente i dati (come kWh erogati e

costi parziali), ma regola attivamente l'intero ciclo di vita dell'operazione, dalla creazione fino al completamento naturale o all'interruzione per emergenza termica.

Per garantire che una ricarica non inizi in condizioni non valide, l'istanziamento è affidata al metodo `factory open()`. Questo metodo esegue una serie di controlli incrociati di dominio: verifica la compatibilità fisica del connettore, controlla che la stazione non sia occupata e delega a `ChargingStation.priceFor()` il calcolo del saldo minimo richiesto, lo stesso metodo che userà poi la strategia di ricarica scelta, così il controllo d'ingresso e l'addebito reale non possono mai calcolare cifre diverse:

Listing 3.4: Apertura di una sessione di ricarica

```
public static ChargingSession open(Driver driver, ChargingStation
    station, ChargingType strategy, Double batteryStart) {

    boolean isAvailable = station.isAvailableFor(driver.
        getConnectorType()) || station.isReservedBy(driver);
    if (!isAvailable) {
        throw new StationNotAvailableException("La colonnina " +
            station.getName() + " non e' disponibile o non e'
            compatibile col tuo veicolo.");
    }

    BigDecimal minimumRequired = station.priceFor(5.0, driver);

    if (!driver.hasSufficientBalance(minimumRequired)) {
        throw new InsufficientBalanceException("Credito
            insufficiente per avviare la sessione. E' richiesto un
            saldo di almeno " +
                minimumRequired + "\\texteuro.");
    }

    return new ChargingSession(driver, station, strategy,
        batteryStart);
}
```

Inoltre, la classe incapsula la logica di avanzamento tramite il metodo `addTick()`, che aggiorna in tempo reale i costi e l'energia immessa, facendo transitare automaticamente lo stato della sessione a completato qualora la batteria raggiunga il 100% della sua capacità.

3.1.4 Finanze

Questo modulo gestisce lo storico dei movimenti economici degli utenti, categorizzandoli tramite l'enumerazione `TransactionType` (ricariche, rimborsi, abbonamenti e depositi).

La scelta architetturale più rilevante per l'entità `Transaction` è stata quella di renderla intrinsecamente immutabile. Trattandosi di dati contabili, era fondamentale garantire che uno storico non potesse essere alterato accidentalmente a runtime. Per questo motivo, la classe è sprovvista di metodi *setter* e i suoi campi sono valorizzabili unicamente al momento della creazione.

Per blindare ulteriormente l'integrità del registro, la generazione di un nuovo movimento non avviene tramite un normale costruttore, ma è protetta da un metodo factory che scarta a priori qualsiasi operazione anomala, come ad esempio i pagamenti a importo zero o privi di causale:

Listing 3.5: Creazione di una transazione contabile

```
public static Transaction create(Driver driver, TransactionType
    type, BigDecimal amount,
                                Double kwh, String description) {

    if (amount == null || amount.compareTo(BigDecimal.ZERO) == 0) {
        throw new IllegalArgumentException("Errore: l'importo non
            puo' essere zero.");
    }
    if (description == null || description.trim().isEmpty()) {
        throw new IllegalArgumentException("Errore: la transazione
            deve avere una descrizione.");
    }
    return new Transaction(driver, type, amount, kwh, description);
}
```

Questa impostazione si è rivelata molto utile per semplificare la logica del livello Business, sapendo di poter fare affidamento su un tracciamento finanziario solido e privo di "transazioni fantasma".

3.1.5 Feedback

A chiudere il Domain Model troviamo la parte dedicata alla qualità del servizio e al supporto. Qui abbiamo deciso di separare l'azione dell'utente (la recensione) dalla conseguente gestione amministrativa (l'allarme per la manutenzione).

Nella classe `Rating`, la creazione di un nuovo feedback è protetta da controlli rigorosi. Abbiamo implementato il metodo factory `leave()`, che non si limita a istanziare la recensione, ma lega quest'ultima alla sessione di ricarica effettiva e impedisce l'inserimento di voti fuori dal range previsto:

Listing 3.6: Creazione di un feedback utente

```
public static Rating leave(Driver driver, ChargingStation station,
    ChargingSession session, Integer stars, String comment) {
    if (stars == null || stars < 1 || stars > 5) {
        throw new InvalidRatingException("Errore di validazione: il
            rating deve essere un valore compreso tra 1 e 5 stelle.
            ");
    }
    return new Rating(driver, station, session, stars, comment);
}
```

Una parte interessante di questo modulo si trova nella classe `RatingAlert`. Quando la media di una stazione cala drasticamente, il sistema genera un avviso. Invece di affidare la

chiusura di questo ticket a un servizio esterno, abbiamo inserito la logica di transizione di stato direttamente all'interno dell'entità. I metodi per sospendere la colonnina o ignorare l'allarme verificano prima che la segnalazione sia effettivamente ancora in sospeso, evitando che un operatore possa inavvertitamente alterare pratiche già chiuse. La decisione di generare l'alert, invece, non spetta al dominio ma al RatingService (vedi paragrafo 3.2.8), che dopo ogni nuova recensione ricalcola la media e verifica la soglia.

Listing 3.7: Chiusura di un alert di manutenzione

```
public void resolveSuspend(String note) {
    if (isPending()) {
        this.status = RatingAlertStatus.RESOLVED_SUSPENDED;
        this.managerNote = note;
    }
}
```

3.2 Business Layer

Il Business Layer rappresenta il livello in cui i dati del Domain Model vengono elaborati e trasformati in funzionalità concrete. L'architettura di questo strato è stata progettata seguendo il principio di singola responsabilità (SRP), suddividendo il codice in servizi specializzati (*Services*) e componenti di simulazione (*Core*).

L'implementazione delle diverse logiche di erogazione (*ChargingStrategy*) verrà approfondita nella successiva sezione dedicata ai Design Pattern.

Di seguito vengono analizzate nel dettaglio le classi centrali che governano e dirigono il flusso dell'applicazione.

3.2.1 GridCluster

Il primo problema tecnico che abbiamo affrontato progettando la simulazione è stato il carico sul database. Poiché il sistema deve ricalcolare i consumi e le temperature della rete ogni pochi secondi, eseguire continuamente query di lettura tramite i DAO avrebbe rallentato l'intera applicazione.

Per risolvere questo collo di bottiglia, abbiamo implementato **GridCluster**. Si tratta di una cache in memoria, gestita come Singleton, che viene popolata una sola volta all'avvio dell'applicazione. Al suo interno, i dati non sono salvati in semplici liste, ma indicizzati strategicamente tramite mappe Hash per garantire tempi di accesso costanti ($O(1)$). Ad esempio, raggruppando le colonnine per ID del trasformatore, il sistema può recuperare l'intero ramo di rete istantaneamente:

```

private Map<Long, List<ChargingStation>> stationsByTransformer;

public void init(TransformerDao tDao, StationDao sDao) {
    this.transformers = tDao.findAll();
    this.allStations = sDao.findAll();

    for (PowerTransformer t : transformers) {
        stationsByTransformer.put(t.getId(), new ArrayList<>())
        ;
    }

    for (ChargingStation station : allStations) {
        Long tId = station.getTransformer().getId();
        if (stationsByTransformer.containsKey(tId)) {
            stationsByTransformer.get(tId).add(station);
        }
    }
}

```

Oltre alla gestione della rete elettrica, questa classe espone anche i metodi di ricerca geografica, utilizzando la distanza euclidea per restituire rapidamente all'utente le colonnine disponibili più vicine.

3.2.2 GridMonitor

Non avendo a disposizione dei sensori fisici che inviano dati telemetrici, avevamo bisogno di un componente che facesse scorrere il tempo in modo autonomo, senza bloccare l'interfaccia grafica.

Abbiamo quindi esteso la classe `Thread` di Java. Un dettaglio implementativo fondamentale è stato impostare questo thread come "Demone" all'interno del costruttore (`this.setDaemon(true);`). In questo modo, il ciclo vitale del monitor viene legato a quello dell'interfaccia JavaFX: se l'utente chiude la finestra dell'applicazione, il thread si arresta in modo pulito senza lasciare processi appesi in background.

Il ciclo di esecuzione principale si sveglia ogni 5 secondi e orchestra le operazioni dei vari servizi, aggregando i dati prima di applicarli:

```

private void onTick() {

    expireShortTermHolds();

    Map<PowerTransformer, Double> heatMap = processActiveSessions()
    ;

    processTransformers(heatMap);

    checkRatingAlerts();
}

```

In particolare, la fase di elaborazione (`processActiveSessions`) calcola il calore generato dalle singole strategie di ricarica in uso (Fast o Eco) e lo somma all'interno di una *Heat Map*, in modo da notificare i trasformatori una sola volta per ciclo, ottimizzando ulteriormente le prestazioni.

3.2.3 LoadBalancerService

Il `LoadBalancerService` è il componente che giustifica l'esistenza del pattern Observer all'interno dell'architettura. Invece di interrogare attivamente la rete per scovare eventuali surriscaldamenti, questo servizio si registra semplicemente in ascolto sui trasformatori.

Quando riceve una notifica di `THERMAL_ALERT`, il servizio reagisce isolando immediatamente il ramo di rete compromesso. L'intervento si divide in due fasi distinte per evitare *deadlock* sul database. Prima viene aperta una transazione esclusiva per forzare lo stato di tutte le colonnine collegate a `OVERLOADED`, impedendo l'avvio di nuove ricariche. Successivamente, la transazione viene chiusa e il servizio delega l'interruzione fisica delle erogazioni già in corso al modulo competente.

Listing 3.8: Gestione dell'alert termico

```
private void handleThermalAlert(PowerTransformer transformer) {

    Connection connection = null;
    try {
        connection = DatabaseManager.getConnection();
        connection.setAutoCommit(false);
        StationDao stationDao = daoFactory.createStationDao(
            connection);

        for (ChargingStation station : stations) {
            station.setOverloaded();
            stationDao.update(station);
        }
        connection.commit();
    } finally {
        closeQuietly(connection); // Previene blocchi a cascata
    }

    for (ChargingSession session : sessionService.getActiveSessions()) {
        if (belongsToTransformer(session.getStation(), stations)) {
            sessionService.forceClose(session);
        }
    }
}
```

È importante notare che questa seconda fase non è atomica nel suo complesso: ogni chiusura forzata (`forceClose`) apre e chiude una propria transazione indipendente, invece di essere racchiusa in un'unica transazione che copra tutte le sessioni del ramo. È una scelta deliberata, non un limite del sistema: se il fallimento di un singolo rimborso facesse

fallire anche gli altri già conclusi con successo, si penalizzerebbero driver che non hanno responsabilità in quel fallimento.

Ogni chiusura forzata produce due effetti economici distinti, salvati nella stessa transazione, il rimborso al Driver e l'accredito allo Station Operator.

Listing 3.9: Rimborso e accredito in `forceClose()`

```
double kwhLastTick = station.getPowerKw() * (5.0 / 3600.0);
BigDecimal refundAmount = BigDecimal.valueOf(
    strategy.calculateCost(kwhLastTick, station, session.getDriver()
    ));

if (wasActive && station.getOperator() != null) {
    double kwhDelivered = session.getKwhDelivered();
    BigDecimal operatorShare = station.getTariffOperator()
        .multiply(BigDecimal.valueOf(kwhDelivered))
        .setScale(2, RoundingMode.HALF_UP);
    operator.addEarnings(operatorShare);
    userDao.update(operator);
}
```

Il Driver viene rimborsato solo del costo dell'ultimo tick da 5 secondi, cioè l'energia pagata ma non ancora erogata al momento dello stacco. Lo Station Operator, allo stesso tempo, viene comunque accreditato della propria quota sull'intera energia realmente erogata fino a quel punto. L'interruzione non annulla quindi l'intera operazione: divide il conto tra le due parti in base a cosa è realmente accaduto prima dello stop.

3.2.4 SessionService

Mentre il Load Balancer gestisce le emergenze, il `SessionService` coordina la normale operatività economica ed energetica. La parte delicata in questa classe è stata garantire una corretta gestione unica di due scritture correlate (sessione e portafoglio) all'interno della stessa transazione

Ogni volta che una ricarica avanza (tramite il metodo `addTick()`) o viene interrotta, il sistema deve aggiornare contemporaneamente la sessione e il portafoglio del Driver. La colonnina invece cambia stato solo all'apertura e alla chiusura, non ad ogni tick.

Invece di delegare la gestione delle connessioni SQL ai singoli DAO, abbiamo centralizzato il controllo transazionale all'interno del Service. Modificando l'impostazione predefinita della connessione (`setAutoCommit(false)`), garantiamo l'atomicità dell'operazione. Se durante il salvataggio dei dati energetici si verifica un errore nel prelievo del denaro dal *Wallet*, l'intera operazione viene annullata tramite `rollback`, assicurando che l'utente non venga mai disallineato rispetto alla colonnina fisica.

Questa scelta progettuale, applicata trasversalmente a tutti i metodi del livello Business (da `openSession` a `forceClose`), ha reso l'architettura estremamente resiliente ai fallimenti hardware o di rete simulati.

3.2.5 StationService

Sebbene gran parte di questo servizio si occupi di gestire il salvataggio e la validazione delle nuove colonnine (delegando i controlli di sicurezza ai Factory Method del dominio), è presente una complessità notevole nella gestione delle prenotazioni (*Short-Term Hold*).

Infatti il problema principale in fase di prenotazione è la concorrenza: due Driver potrebbero tentare di riservare la stessa stazione nell'esatto momento in cui si libera. Per prevenire *race condition*, il servizio non si limita a leggere lo stato della colonnina e aggiornarlo in memoria, ma sfrutta i lock del database. Richiamando un'operazione di aggiornamento atomico tramite il DAO (`acquireAtomicHold`), il servizio garantisce che solo la prima transazione vada a buon fine, bloccando in modo sicuro e definitivo la risorsa e lanciando una `StationNotAvailableException` per il secondo utente.

3.2.6 Sicurezza e Validazione

La responsabilità di sicurezza dell'applicazione è divisa in due aree distinte, gestite da servizi separati:

- **Autenticazione (AuthService):** Il sistema non salva mai le credenziali in chiaro. L'hashing delle password è delegato alla libreria esterna **BCrypt**. Inoltre, il flusso di registrazione varia dinamicamente in base al ruolo richiesto, imponendo l'assegnazione di coordinate GPS iniziali esclusivamente ai Driver.
- **Collaudo dell'Infrastruttura (ValidationService):** Nessuna stazione registrata da un operatore entra in rete automaticamente. Prima dell'approvazione, l'Energy Manager avvia un collaudo software per intercettare anomalie hardware.

È proprio quest'ultimo punto a evitare gli errori di inserimento. Ecco come il sistema "interroga" la colonnina prima di permetterne l'attivazione:

Listing 3.10: Test di validazione di una stazione

```
public boolean testStation(ChargingStation station) {  
  
    if (station.getPowerKw() > station.getConnectorType().  
        getMaxPowerKw()) {  
        System.err.println("Test fallito: Potenza eccessiva per il  
            tipo di connettore.");  
        return false;  
    }  
  
    return simulateConnectivityTest(station.getLatitude(), station.  
        getLongitude());  
}
```

Solo se il metodo restituisce `true`, il servizio procede ad aprire la transazione sul database, fissando la tariffa della piattaforma e cambiando lo stato della stazione in `ACTIVE`.

3.2.7 Gestione Utente e Portafoglio

La gestione del profilo utente e del suo portafoglio è affidata al `DriverService` e al `WalletService`. Quest'ultimo, in particolare, implementa le funzionalità legate ai pagamenti e ai piani tariffari, sollevando la classe di dominio `Driver` dalla responsabilità di interfacciarsi con il database.

Una delle operazioni più delicate di questo modulo è il cambio di abbonamento, la cui logica di calcolo è già contenuta in `Driver.updatePlan()` (vedi paragrafo 3.1.1). Il `WalletService`, prima di delegare al dominio, effettua un'unica verifica aggiuntiva sul saldo:

Listing 3.11: Verifica del saldo prima dell'upgrade

```
if (delta.compareTo(BigDecimal.ZERO) > 0
    && driver.getWalletBalance().compareTo(delta) < 0) {
    throw new IllegalStateException(
        "Fondi insufficienti: servono EUR " + delta + " per l'
        upgrade a " + plan.name() + ".");
}
BigDecimal charged = driver.updatePlan(plan);
```

3.2.8 Gestione Feedback e Manutenzione

Il flusso di controllo della qualità del servizio è interamente gestito dal `RatingService`. Il ciclo di vita di un feedback inizia nel momento in cui il `Driver` conclude una ricarica e decide di valutare l'esperienza.

In questa prima fase, il sistema esegue un duplice controllo. Dapprima verifica a livello database che l'utente non abbia già recensito quella specifica sessione; superato questo filtro, delega la validazione del formato del voto (da 1 a 5 stelle) al dominio.

La parte interessante e funzione aggiuntiva di questa componente risiede nella logica che si innesca subito dopo il salvataggio. Infatti, non essendoci un controllo umano costante sulle singole colonnine, il servizio ricalcola in tempo reale la media della stazione appena recensita. Se tale parametro scende sotto la soglia critica di 2.0 (con un campione minimo di 5 valutazioni), il sistema interviene autonomamente generando un `RatingAlert`.

Listing 3.12: Ricalcolo della media e alert di manutenzione

```
private void checkRatingAlert(Connection connection,
    ChargingStation station) {

    ratingDao.recalculateAverage(station.getId());
    ChargingStation updatedStation = stationDao.findById(station.
        getId());

    Double currentAvg = updatedStation.getAverageRating();
    Integer totalRatings = updatedStation.getTotalRatings();

    if (currentAvg != null && currentAvg < 2.0 && totalRatings !=
        null && totalRatings >= 5) {
        if (!ratingAlertDao.existsOpenAlertForStation(station.getId()
            ())) {
            RatingAlert alert = RatingAlert.create(updatedStation,
                currentAvg);
            ratingAlertDao.save(alert);
        }
    }
}
```

Una volta ricevuta la segnalazione sulla propria dashboard, sarà discrezione dell'Energy Manager decidere se ignorarla (qualora si trattasse di un falso allarme) o sospendere la stazione difettosa.

Nota: la mappa di Firenze e il Jitter

Per rendere i test più verosimili, abbiamo deciso di utilizzare come ambiente geografico l'area urbana di Firenze, mappando i veri quartieri (dal Centro Storico alla periferia) tramite l'enumerazione *Zone*.

Ci siamo però accorti subito di un problema: registrando più colonnine nello stesso quartiere, queste avrebbero assunto le stesse identiche coordinate di base, finendo per sovrapporsi perfettamente in caso di futura rappresentazione grafica. Per risolverlo, abbiamo introdotto una piccola funzione di *Jitter* all'interno dell'enum. Questo metodo calcola un microscopico scostamento casuale (corrispondente a circa 500 metri) e lo applica al centro esatto della zona, in modo tale da generare latitudini e longitudini leggermente diverse per ogni stazione, pur rimanendo all'interno del quartiere di riferimento.

Listing 3.13: Distribuzione geografica con jitter

```
public double[] resolveWithJitter() {

    double jLat = (Math.random() - 0.5) * 0.01;
    double jLng = (Math.random() - 0.5) * 0.01;
    return new double[]{ baseLat + jLat, baseLng + jLng };
}
```

3.3 Controllers

Ogni schermata dell'applicazione ha un Controller associato, il cui compito è sempre lo stesso: leggere cosa ha fatto l'utente, per esempio un click o un testo inserito, passare la richiesta al Service giusto del Business Layer, e mostrare il risultato a schermo. Per questo motivo la struttura interna dei Controller è quasi identica tra loro, e di seguito ne presentiamo l'elenco raggruppato per area applicativa, con una breve descrizione del ruolo di ciascuno.

Autenticazione

- **LoginController:** Raccoglie email e password e delega l'autenticazione ad **AuthService**, instradando l'utente verso la dashboard corretta in base al ruolo.
- **RegistrationController:** Raccoglie i dati per un nuovo account, incluso il ruolo scelto (Driver, Station Operator, Energy Manager).

Area Driver

- **DriverMenuController:** Contenitore della dashboard Driver: gestisce il menu laterale e sostituisce solo il contenuto centrale ad ogni navigazione.
- **StationExplorerController:** Mostra le colonnine compatibili nelle vicinanze e permette di prenotarne una o avviare direttamente una sessione.
- **LiveSessionController:** Mostra l'avanzamento in tempo reale di una ricarica in corso, aggiornandosi ad ogni tick del **GridMonitor**.
- **ChargingHistoryController:** Elenca le sessioni passate del Driver, completate o interrotte per allarme termico.
- **FundWalletController:** Gestisce la ricarica del saldo del wallet.
- **AccountController:** Mostra i dati del profilo: piano di abbonamento (modificabile), email e veicolo (in sola lettura).

Area Station Operator

- **OperatorMenuController;** Dashboard riepilogativa dell'operatore, con lo stato delle sue colonnine e i guadagni totali.
- **NewStationController:** Modulo di registrazione di una nuova colonnina, con i suoi parametri tecnici.
- **StationHubController:** Elenco dettagliato delle colonnine possedute dall'operatore, con stato operativo di ciascuna (online, in sovraccarico, ecc.) e accesso rapido alla registrazione di una nuova.
- **FinancialReportsController:** Riepilogo economico dell'operatore: guadagno totale, energia venduta e numero di sessioni erogate.

Area Energy Manager

- **EnergyManagerMenuController:** Contenitore della dashboard dell'Energy Manager, da cui si accede alle altre schermate di gestione della rete.
- **ApprovalQueueController:** Coda delle colonnine in attesa di verifica: l'Energy Manager può approvarle o rifiutarle.
- **RatingIncidentManagerController:** Elenco degli alert generati automaticamente quando una colonnina riceve valutazioni troppo basse, con le azioni di sospensione o chiusura dell'alert.
- **ReliabilityDashboardController:** Stato in tempo reale della rete elettrica: carico e temperatura di ogni trasformatore, con evidenza visiva di eventuali allarmi termici.

Un dettaglio implementativo interessante si trova in `FinancialReportsController`: prima di leggere il guadagno totale dell'operatore, il controller non si fida del valore tenuto in sessione, ma lo rilegge direttamente dal database.

Listing 3.14: Aggiornamento del controller con dati freschi

```
User refreshed = NavigationManager.getAuthService().refreshUser(
    loggedOperator.getId());
if (refreshed instanceof StationOperator op) {
    this.loggedOperator = op;
}
BigDecimal revenue = loggedOperator.getTotalEarnings() != null
    ? loggedOperator.getTotalEarnings() : BigDecimal.ZERO;
```

L'oggetto `User` mantenuto in sessione resta valido per identità e ruolo, ma qualunque valore in continua variazione, come per esempio il guadagno totale, aggiornato ad ogni sessione chiusa da un `Driver` — deve essere riletto dal Service competente all'apertura della schermata, invece di fidarsi di una copia che potrebbe non aggiornata.

3.4 ORM

Come già detto in precedenza il livello DAO si occupa di tradurre le righe delle tabelle SQL in oggetti Java e viceversa. Il `GenericDao<T>` definisce le operazioni base che garantiscono la persistenza dei dati:

Listing 3.15: Interfaccia DAO generica

```
public interface GenericDao<T> {
    void save(T entity);
    T findById(Long id);
    List<T> findAll();
    void update(T entity);
    void delete(Long id);
}
```

Ogni entità del dominio ha poi una classe concreta (es. `PostgresUserDao`) che implementa queste operazioni scrivendo query SQL vere tramite `PreparedStatement`, ed esegue la trasformazione in entrambe le direzioni: dall'oggetto Java verso le colonne della tabella al momento del salvataggio, e dalle colonne del `ResultSet` di nuovo verso un oggetto Java al momento della lettura.

Tabella users

La tabella `users` contiene sia i Driver che gli Station Operator che gli Energy Manager, distinti da una colonna `role`. Il mapping deve quindi decidere, riga per riga, quale sottoclasse istanziare:

Listing 3.16: Mapping utente da ResultSet

```
private User extractUserFromResultSet(ResultSet rs) throws
SQLException {
    Long id = rs.getLong("id");
    String name = rs.getString("name");
    String dbEmail = rs.getString("email");
    String password = rs.getString("password");
    Role role = Role.valueOf(rs.getString("role"));

    if (role == Role.DRIVER) {
        Double lat = rs.getDouble("latitude"); if (rs.isNull())
            lat = null;
        Double lon = rs.getDouble("longitude"); if (rs.isNull())
            lon = null;
        Double cap = rs.getDouble("battery_capacity"); if (rs.
            wasNull()) cap = null;
        String connTypeStr = rs.getString("connector_type");
        var cType = connTypeStr != null ? ConnectorType.valueOf(
            connTypeStr) : null;
        String subPlanStr = rs.getString("subscription_plan");
        var sPlan = subPlanStr != null ? SubscriptionPlan.valueOf(
            subPlanStr) : null;
        BigDecimal balance = rs.getBigDecimal("wallet_balance");
        if (balance == null) balance = BigDecimal.ZERO;
        return Driver.reconstitute(id, name, dbEmail, password, lat
            , lon, cType, sPlan, cap, balance);

    } else if (role == Role.STATION_OPERATOR) {
        BigDecimal earnings = rs.getBigDecimal("total_earnings");
        if (earnings == null) earnings = BigDecimal.ZERO;
        return StationOperator.reconstitute(id, name, dbEmail,
            password, earnings);

    } else if (role == Role.ENERGY_MANAGER) {
        return EnergyManager.reconstitute(id, name, dbEmail,
            password);
    }
    return null;
}
```

Colonne come `wallet_balance` o `total_earnings` esistono nella tabella ma hanno senso solo per un tipo di utente: per un Energy Manager, ad esempio, restano ovviamente NULL. Il DAO legge la colonna `role` e usa quel valore per capire quale oggetto Java ricostruire, e quindi come debba muoversi.

Metodo `reconstitute()`

Il mapping non chiama mai `new Driver(...)` direttamente, ma un metodo chiamato `reconstitute()`. La differenza è importante: quando un Driver viene creato per la prima volta (in fase di registrazione) deve passare da un Factory Method che valida i

dati in ingresso, perché in quel momento i dati arrivano da un utente e potrebbero essere sbagliati. Quando invece un Driver viene riletto dal database, i dati sono già stati validati in passato (e protetti da vincoli SQL come i CHECK dello schema) — rivalidarli ad ogni lettura sarebbe inutile. `reconstitute()` si limita quindi a ricostruire l'oggetto così com'è, senza ripetere controlli già superati.

Ricostruzione parziale (stub)

A volte il DAO non ha bisogno di ricostruire un oggetto per intero, ma solo di una sua parte. Nel recupero di un `RatingAlert`, ad esempio, non serve caricare l'intera colonnina collegata (tariffe, stato, coordinate...): basta sapere il suo nome, da mostrare nella schermata dell'Energy Manager.

Listing 3.17: Ricostruzione parziale di `ChargingStation`

```
ChargingStation station = ChargingStation.reconstitute(
    stId, null, null, stName, null, null, null, null, null,
    null,
    null, null, null, null, null, null, null
);
return RatingAlert.reconstitute(alertId, station, avgAtCreation,
    status, managerNote, createdAt);
```

Questa `ChargingStation` è effettivamente un oggetto reale (viene chiamato "stub"), ma della quale vengono inseriti solo i campi necessari, mentre il resto è `null`. In questo modo si ottiene maggiore efficienza, infatti evita una query aggiuntiva per recuperare dati che non verrebbero comunque mostrati, tuttavia nel caso in futuro venisse ripassato a un metodo che si aspetta una colonnina completa (ad esempio per un aggiornamento), andrebbe prima ricaricato per intero.

I vincoli dello schema

Oltre alla validazione fatta dal codice Java, anche lo schema del database applica i propri controlli, indipendentemente da cosa succede nel Business Layer:

Listing 3.18: Vincoli SQL di integrità

```
wallet_balance DECIMAL(10, 2) CHECK (wallet_balance IS NULL OR
    wallet_balance >= 0),
...
stars INTEGER NOT NULL CHECK (stars >= 1 AND stars <= 5),
...
CONSTRAINT uq_driver_session UNIQUE (driver_id, session_id)
```

Se per un errore di programmazione un service riuscisse a costruire un saldo negativo o un voto fuori dal range 1-5, sarebbe comunque il database a rifiutare l'inserimento. Può essere visto come un secondo controllo: nel caso ci sia qualcosa che non vada bene nel codice Java, questo non riuscirebbe comunque a produrre dati incoerenti nel database.

3.5 Idiomi e Design Pattern

Per risolvere alcuni problemi e per rendere il codice aperto per un eventuale estensione futura, sono stati usati alcuni design pattern.

3.5.1 Observer Pattern

Quando un trasformatore (`PowerTransformer`) si scalda troppo si limita a lanciare un avviso, mentre chi lo riceve e decide cosa fare è `LoadBalancerService`.

Listing 3.19: Interfacce Subject e Observer

```
public interface Subject {
    void attach(Observer observer);
    void detach(Observer observer);
    void notifyObservers(TransformerEvent event);
}

public interface Observer {
    void update(Subject source, TransformerEvent event);
}

public enum TransformerEvent {
    THERMAL_ALERT,
    COOLING_COMPLETE
}
```

Il vantaggio è che i due componenti restano indipendenti, se per esempio volessimo aggiungere un secondo componente che si limita a registrare gli allarmi, basterebbe aggiungere agli ascoltatori anche lui, senza dover modificare il codice del trasformatore.

3.5.2 Strategy Pattern

Un Driver può scegliere tra due modalità di ricarica, Fast o Eco, e ciascuna ha un proprio modo di calcolare il costo e il calore prodotto. Invece che usare un `if/else`, che chiaramente sarebbe dovuta essere modificata nel caso di un'aggiunta futura, si è definita un'unica interfaccia comune che poi ogni modalità implementa diversamente. In questo modo `SessionService` si interfaccia senza sapere quale modalità di ricarica ci sia dietro.

Listing 3.20: Interfaccia delle strategie di ricarica

```
public interface ChargingStrategy {
    double calculateCost(double kwh, ChargingStation station,
        Driver driver);
    double getHeatIncrement();
    ChargingType getType();
}
```

3.5.3 Factory Method

Per evitare che alcuni oggetti possano essere creati pur non avendo rispettato determinate regole, la creazione di un oggetto deve passare sempre da un metodo che prima controlla che tutto sia in regola, e solo dopo costruisce l'oggetto:

Listing 3.21: Factory Method per la creazione del rating

```
public static Rating leave(Driver driver, ChargingStation station,
    ChargingSession session,
    Integer stars, String comment) {
    if (stars == null || stars < 1 || stars > 5) {
        throw new InvalidRatingException("Errore di validazione: il
            rating deve essere un
            valore compreso tra 1 e 5 stelle.");
    }
    return new Rating(driver, station, session, stars, comment);
}
```

Lo stesso principio si ritrova quando si apre una sessione di ricarica, quando si registra una nuova colonnina o quando si crea un alert di qualità: in tutti questi casi il controllo e la creazione avvengono insieme, così è impossibile ottenere un oggetto "a metà", che esiste ma è in uno stato che non dovrebbe essere permesso.

Listing 3.22: altro esempio di Factory method, qui è usato per creare alert di rating. A differenza degli esempi precedenti, qui il costruttore non deve validare nulla, i dati arrivano già controllati da chi lo invoca, ma resta comunque privato per impedire di costruire un alert in uno stato diverso da PENDING.

```
private RatingAlert(ChargingStation station, Double
    avgAtCreation) {
    this.station = station;
    this.avgAtCreation = avgAtCreation;
    this.status = RatingAlertStatus.PENDING;
    this.managerNote = null;
    this.createdAt = LocalDateTime.now();
}

public static RatingAlert create(ChargingStation station, Double
    avgAtCreation) {
    return new RatingAlert(station, avgAtCreation);
}
```

3.5.4 Singleton

Il sistema tiene traccia di quali colonnine sono collegate a quale trasformatore. Questa informazione viene consultata ogni 5 secondi dal componente che simula il passare del tempo `GridMonitor` e ogni volta che un trasformatore si surriscalda dal componente che deve bloccare le colonnine.

Se questa informazione fosse duplicata in due copie diverse, si rischierebbe un disallineamento tra i due componenti. La soluzione è garantire che ne esista una sola copia in tutto il programma, caricata una volta sola all'avvio e poi consultata da tutti:

Listing 3.23: Singleton GridCluster per la cache della rete

```
public class GridCluster {

    private static GridCluster instance;

    private List<PowerTransformer> transformers;
    private Map<Long, List<ChargingStation>> stationsByTransformer;
    private List<ChargingStation> allStations;

    private GridCluster() {
        this.transformers = new ArrayList<>();
        this.stationsByTransformer = new HashMap<>();
        this.allStations = new ArrayList<>();
    }

    public static synchronized GridCluster getInstance() {
        if (instance == null) {
            instance = new GridCluster();
        }
        return instance;
    }

    public void init(TransformerDao tDao, StationDao sDao) {
        this.transformers = tDao.findAll();
        this.allStations = sDao.findAll();
        for (PowerTransformer t : transformers) {
            stationsByTransformer.put(t.getId(), new ArrayList<>());
        }
        for (ChargingStation station : allStations) {
            Long tId = station.getTransformer().getId();
            if (stationsByTransformer.containsKey(tId)) {
                stationsByTransformer.get(tId).add(station);
            }
        }
    }
}
```

La parola `synchronized` su `getInstance()` serve perché più componenti girano in parallelo su thread diversi: senza questa protezione, due richieste arrivate nello stesso istante potrebbero entrambe trovare `instance` ancora nulla e crearne due copie per errore. Da notare inoltre che i dati non vengono letti dal database ad ogni richiesta: `init()` viene chiamato una sola volta all'avvio dell'applicazione, e da quel momento in poi `getStationsForTransformer()` risponde all'istante leggendo dalla mappa già in memoria, senza interrogare più il database.

Capitolo 4

Testing

4.1 Strategia di Verifica

I test sono eseguiti su due livelli:

- **Livello 1 – Business Logic:** Service, componenti core (`GridCluster`, `GridMonitor`) e Strategy, testati isolando ogni dipendenza esterna con Mockito.
- **Livello 2 — Persistence Layer:** i DAO Postgres, testati contro un database H2 reale in modalità `MODE=PostgreSQL`, per verificare che le query SQL scritte a mano funzionino davvero e non solo che le interazioni tra oggetti siano quelle attese.

4.1.1 Tecnologie e Strumenti

- **JUnit 5:** framework di esecuzione, con le annotazioni `@BeforeEach/@AfterEach` per il setup e cleanup di ogni test e `@BeforeAll/@AfterAll` per le risorse costose (come la connessione H2) condivise da un'intera classe.
- **Mockito:** usato non solo per mockare interfacce (`StationDao`, `SessionService`, ...) ma anche per mockare **metodi statici** tramite `Mockito.mockStatic()` — una necessità specifica di questo progetto, perché sia `DatabaseManager.getConnection()` sia `GridCluster.getInstance()` sono chiamate statiche che altrimenti aprirebbero una connessione reale o toccherebbero lo stato globale del Singleton durante un test unitario.
- **H2 in modalità `MODE=PostgreSQL`:** usato esclusivamente nei test dei DAO, con lo schema reale caricato all'avvio tramite `INIT=RUNSCRIPT FROM './src/main/resources/scheme.` — lo stesso file `scheme.sql` usato in produzione, quindi i test verificano le query contro la struttura di tabelle reale, non una sua approssimazione.

4.2 Livello 1: Business Logic

4.2.1 Logica pura, senza mock — `ChargingStrategiesTest`

Le strategie di ricarica (`FastChargingStrategy`, `EcoChargingStrategy`) hanno stesso input, stesso output e nessuna dipendenza esterna. Per questo motivo il loro test non usa

Mockito: costruisce oggetti di dominio reali con `reconstitute()` e confronta il risultato con un secondo calcolo indipendente, scritto direttamente nel test:

```
private double basePrice(double kwh, SubscriptionPlan plan) {
    double discount = (plan != null) ? plan.getDiscount() : 0.0;
    BigDecimal platAfter = TARIFF_PLAT.multiply(BigDecimal.ONE.
        subtract(BigDecimal.valueOf(discount)));
    BigDecimal rate = TARIFF_OP.add(platAfter);
    return rate.multiply(BigDecimal.valueOf(kwh)).setScale(2,
        RoundingMode.HALF_UP).doubleValue();
}

@Test
void testEcoCalculateCostWithSubscriptionDiscount() {
    double kwh = 100.0;
    Driver premiumDriver = testDriver(3L, "Anna", "anna@test.com",
        SubscriptionPlan.PREMIUM);
    double expectedPremium = round2(basePrice(kwh, SubscriptionPlan.
        PREMIUM) * 0.90);
    assertEquals(expectedPremium, ecoStrategy.calculateCost(kwh,
        mockStation, premiumDriver), 0.001,
        "PREMIUM: sconto piano sulla sola quota piattaforma,
        poi Eco 10%");
}
```

Riscrivere la formula nel test invece di limitarsi a un valore atteso hardcoded (es. `assertEquals(33.30, ...)`) ha un vantaggio: se le tariffe di test cambiano, il valore atteso si ricalcola automaticamente insieme a esse, e il test continua a verificare la *relazione* tra i numeri (lo sconto si applica solo alla quota piattaforma) invece di un singolo numero fisso che potrebbe diventare scorrelato dalla logica reale.

4.2.2 Singleton e reset dello stato : GridClusterTest

`GridCluster` è un Singleton (Capitolo 3.5 – Idiomi e Design Pattern): la sua istanza sopravvive tra un test e l'altro se non viene esplicitamente distrutta, col rischio che i dati caricati da un test "contaminino" il test successivo. La soluzione adottata è azzerare il campo statico `instance` via reflection prima di ogni test:

```
@BeforeEach
void setUp() throws Exception {
    Field instanceField = GridCluster.class.getDeclaredField("
        instance");
    instanceField.setAccessible(true);
    instanceField.set(null, null);

    gridCluster = GridCluster.getInstance();
    transformerDaoMock = Mockito.mock(TransformerDao.class);
    stationDaoMock = Mockito.mock(StationDao.class);
}
```

Con l'istanza garantita pulita, il test su `getNearestAvailable()` verifica sia la logica di filtro (solo le colonnine `ACTIVE`) sia quella di ordinamento (per distanza crescente), usando dati costruiti ad hoc invece di un vero calcolo GPS:

```
List<ChargingStation> nearest = gridCluster.getNearestAvailable(0.0, 0.0);

assertEquals(2, nearest.size(), "Deve escludere le stazioni non ACTIVE");
assertEquals("Vicino", nearest.get(0).getName(), "La prima deve essere quella geometricamente piu vicina");
assertEquals("Lontano", nearest.get(1).getName(), "La seconda deve essere quella piu lontana");
```

4.2.3 Service e gestione transazionale : `DriverServiceTest`, `LoadBalancerServ`

I Service che scrivono sul database aprono e chiudono esplicitamente una transazione JDBC (`setAutoCommit(false)`, poi `commit()` o `rollback()` in caso di errore, sempre seguito da `close()`). Testare questo comportamento richiede di intercettare la connessione prima che il Service la apra davvero — ed è qui che entra in gioco il mock statico:

Listing 4.1: Mock statico di `DatabaseManager` per intercettare la connessione JDBC.

```
@BeforeEach
void setUp() throws SQLException {
    connectionMock = mock(Connection.class);
    mockedDbManager = mockStatic(DatabaseManager.class);
    mockedDbManager.when(DatabaseManager::getConnection).thenReturn(
        (connectionMock);

    daoFactoryMock = mock(DaoFactory.class);
    userDaoMock = mock(UserDao.class);
    when(daoFactoryMock.createUserDao(connectionMock)).thenReturn(
        userDaoMock);

    driverService = new DriverService(daoFactoryMock);
}

@AfterEach
void tearDown() {
    if (mockedDbManager != null) {
        mockedDbManager.close(); // fondamentale: rilascia il mock statico globale
    }
}
```

Con la connessione sotto controllo, si possono verificare separatamente il percorso di successo e quello di fallimento — due test speculari che verificano esattamente le proprietà opposte sulla stessa transazione:

```

@Test
void testUpdateDriverLocation_Success() throws SQLException {
    Driver mockDriver = mock(Driver.class);
    driverService.updateDriverLocation(mockDriver, 43.7696,
        11.2558);

    verify(mockDriver, times(1)).updatePosition(43.7696, 11.2558);
    verify(userDaoMock, times(1)).update(mockDriver);
    verify(connectionMock, times(1)).commit();
    verify(connectionMock, times(1)).close();
}

```

```

@Test
void testUpdateDriverLocation_Failure() throws SQLException {
    doThrow(new RuntimeException("Database error")).when(
        userDaoMock).update(any(Driver.class));

    assertThrows(RuntimeException.class, () ->
        driverService.updateDriverLocation(mockDriver, 0.0, 0.0));

    verify(connectionMock, times(1)).rollback();
    verify(connectionMock, times(1)).close();
    verify(connectionMock, never()).commit();
}

```

Questa coppia di test è particolarmente significativa perché non si limita a verificare "il metodo non lancia errori": verifica che, in caso di eccezione, il sistema non lasci una transazione a metà — un dettaglio che un test puramente Black-Box (solo sul valore di ritorno) non potrebbe cogliere.

`LoadBalancerServiceTest` applica la stessa tecnica ma ne combina due insieme, mockando sia `DatabaseManager` sia il Singleton `GridCluster` nello stesso test, per isolare completamente la reazione a un evento `THERMAL_ALERT`:

Listing 4.2: Doppio mock statico: connessione JDBC e Singleton `GridCluster`.

```

mockedDbManagerStatic = mockStatic(DatabaseManager.class);
mockedDbManagerStatic.when(DatabaseManager::getConnection).
    thenReturn(connectionMock);

gridClusterMock = mock(GridCluster.class);
mockedGridClusterStatic = mockStatic(GridCluster.class);
mockedGridClusterStatic.when(GridCluster::getInstance).thenReturn(
    gridClusterMock);

```

Il test sul `THERMAL_ALERT` verifica in un colpo solo tre effetti attesi del pattern Observer descritto nel Capitolo 3.5: che la colonnina venga marcata come sovraccarica, che la modifica sia persistita, e che la chiusura delle sessioni attive sia *delegata* a `SessionService` invece di essere gestita internamente:

```
loadBalancerService.update(transformerMock, TransformerEvent.
    THERMAL_ALERT);

verify(stationMock, times(1)).setOverloaded();
verify(stationDaoMock, times(1)).update(stationMock);
verify(connectionMock, times(1)).commit();
verify(sessionServiceMock, times(1)).forceClose(sessionMock);
```

L'ultima riga è la più importante: verifica che `LoadBalancerService` non sappia *come* si chiude una sessione (rimborso, aggiornamento wallet), sappia solo *che* va chiusa — la stessa separazione di responsabilità già discussa a proposito di `LoadBalancerService` nel capitolo sul Business Layer.

4.3 Livello 2: Persistence Layer (DAO)

I test dei DAO non usano mock: interrogano un database H2 reale, inizializzato ad ogni classe di test con lo stesso `schema.sql` usato in produzione.

Listing 4.3: Avvio di H2 con lo schema reale di produzione.

```
@BeforeAll
static void startDatabase() throws SQLException {
    String url = "jdbc:h2:mem:chargenet_test;MODE=PostgreSQL;
        DATABASE_TO_LOWER=TRUE;"
        + "DEFAULT_NULL_ORDERING=HIGH;INIT=RUNSCRIPT FROM
        './src/main/resources/schema.sql'";
    connection = DriverManager.getConnection(url, "sa", "");
}
```

Poiché la connessione è condivisa da tutta la classe (`@BeforeAll`, non `@BeforeEach`, per non pagare il costo di riavviare H2 ad ogni singolo test), ogni test deve invece garantire di ripartire da una tabella vuota. Questo pone un problema: la tabella `charging_sessions` ha una foreign key verso `charging_stations`, quindi uno svuotamento diretto verrebbe rifiutato dal database finché non si disattivano temporaneamente i vincoli:

Listing 4.4: Pulizia delle tabelle rispettando temporaneamente i vincoli di integrità.

```
try (Statement stmt = connection.createStatement()) {
    stmt.execute("SET REFERENTIAL_INTEGRITY FALSE");

    stmt.execute("TRUNCATE TABLE charging_sessions RESTART IDENTITY
");
    stmt.execute("TRUNCATE TABLE charging_stations RESTART IDENTITY
");
    stmt.execute("TRUNCATE TABLE users RESTART IDENTITY");
    stmt.execute("TRUNCATE TABLE power_transformers RESTART
IDENTITY");

    // ... reinserimento dei dati minimi necessari (operatore,
        driver, trasformatore, stazione) ...

    stmt.execute("SET REFERENTIAL_INTEGRITY TRUE");
}
```

Il vantaggio di questo approccio rispetto a un semplice `DELETE FROM: RESTART IDENTITY` azzerava anche i contatori auto-incrementali, così ogni test parte con ID prevedibili

Capitolo 5

Demo